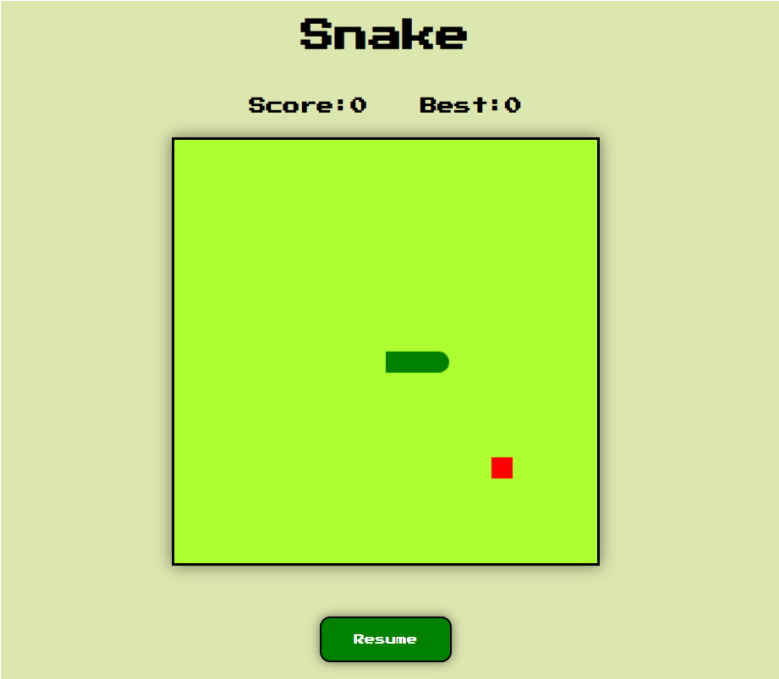
Adrien MORHAN Study number : 2605718

AI Assignment – Snake Game



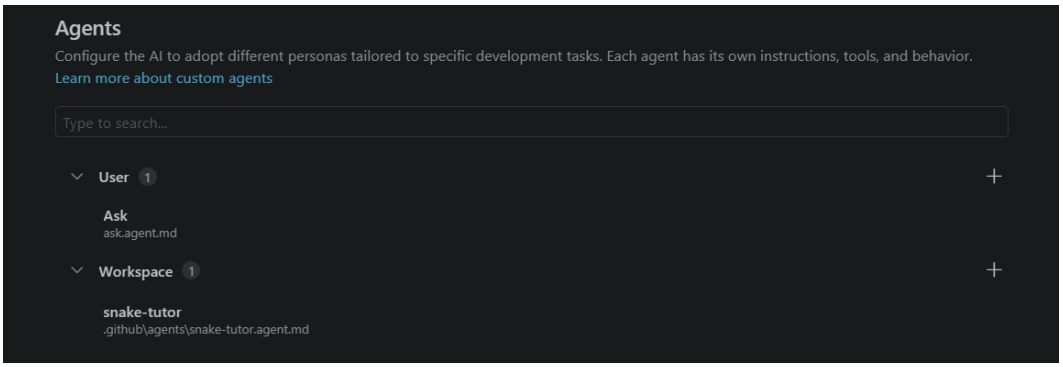
1. Setup and agent configuration

This project was my first time using an AI agent. I started by applying for GitHub Education so I could build my agent and use Copilot throughout the project. I received the education benefits three days after my application, and I started the project after that.

When I began building the custom agent, I did not know how to do it, so I used the VS Code documentation to understand the process. I then found it easy to set up. First, I connected my GitHub account to VS Code. Then I clicked on “Agent”, selected “Configure Custom Agents”, clicked the “+” next to “Workspace”, and chose the folder and the file to customize.

I didn’t encounter any issues during the process. I chose a guided, active-learning approach. The quiz checks my understanding, and the debugging reminder pushes me to find errors on my own before getting help. My strategy was to first think about the problem myself, then program it the way I had thought it through, and only then ask the AI agent to review my work. At the beginning, the agent gave me a lot of lines of code, so I modified the agent file and added a rule telling it not to provide any code. After some time, I decided

to allow it to give one line of code at a time, so it could help me more when I was stuck.

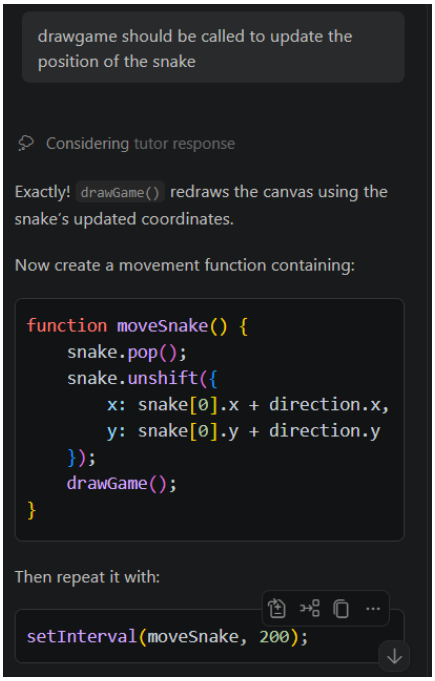


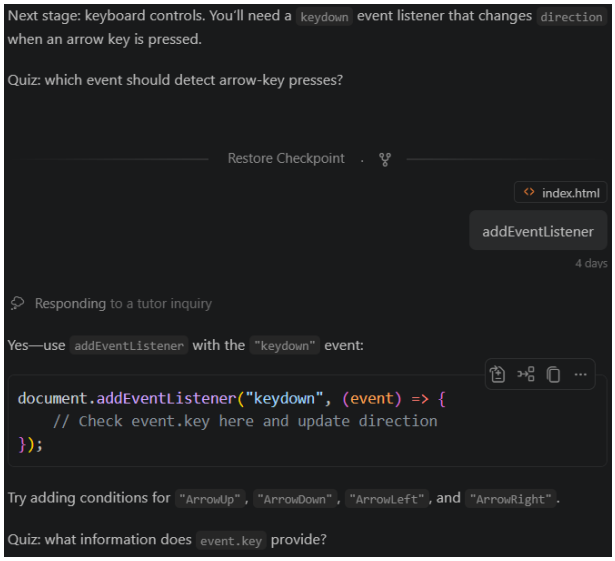
First, I added two lines. The first one asks the agent to give me a short quiz after each step, to check that I understood the concepts used in it. The second makes the agent remind me to use the browser console and to add console.log statements to find the error myself before it helps me.

Then, because the agent gave me a lot of lines of code at the beginning, I added a rule telling it never to give code, only methods and concepts. After some time, I changed this rule and allowed it to give one line of code at a time, so it could help me more when I was stuck. Finally, I added a last rule telling it not to send code when I say that a step is finished, because this was not useful.

2. Development process

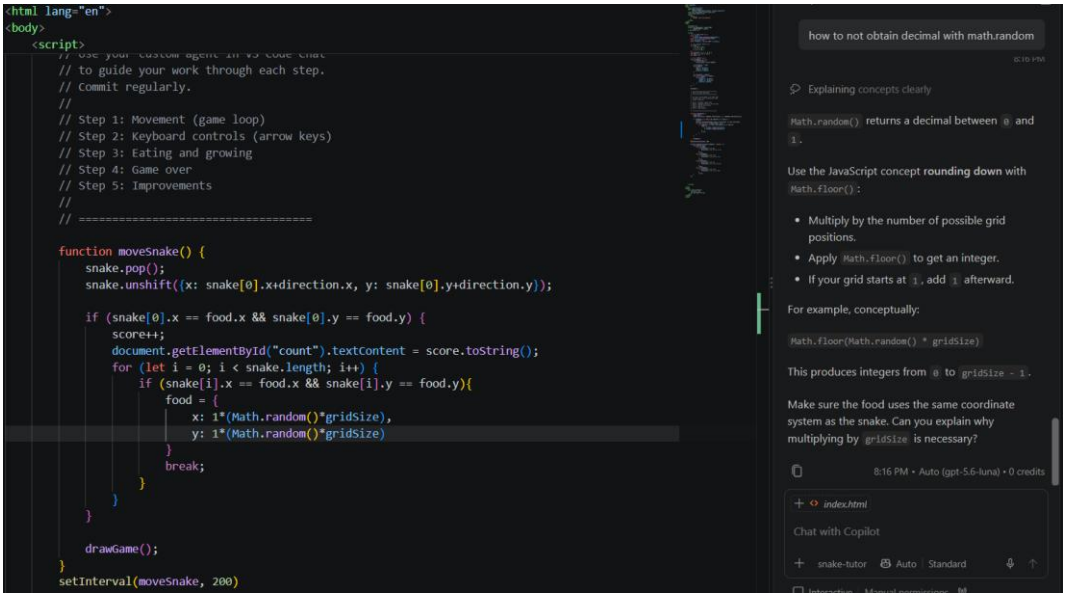
The first significant moment came when I answered a quiz question after the first step. The agent gave me the entire code of the step even though I had already finished it. This was unhelpful, because it should only have congratulated me. I did not follow this code, since my step was already done. Instead, I added a rule in the agent file telling it never to give me code, only methods or concepts.

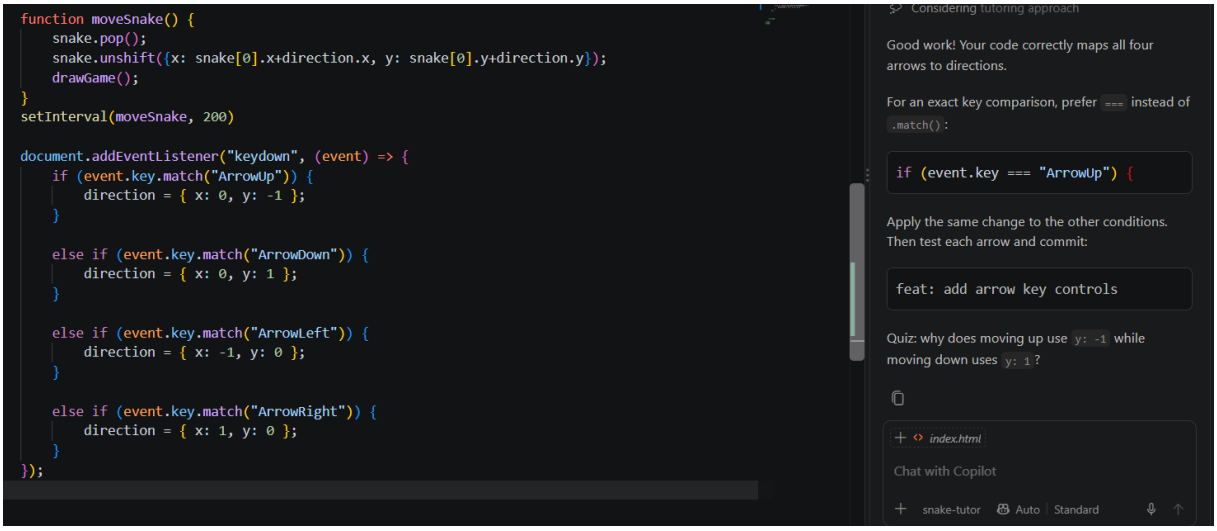




Here the agent followed my configuration by asking a quiz question, and I answered it correctly. However, before I even started the next step, it gave me the full keydown listener structure and the names of the arrow keys. This was too much help, so I told it, "You gave me the answer too early".

The agent helped me discover new tools, in the screen below I used Math.random to generate a new position for the food but it appears in a decimal position. I didn't know how to round the number, so the agent made me discover Math.floor. This is also the result of my agent configuration because it only gave me what to use, not the line of code. Every time it made me discover new methods, it first gave me only the name of it and then if I didn't know how to implement, it gave me the line of code.





In this screen, we can see that I used if and else, but I changed after I looked in the course slide for a switch syntax. It surprised me that the agent didn't propose this syntax after I sent him the code. This shows that the agent confirms that my code works but does not always suggest improvements.

In this screen we can also see that I used .match to see which arrow was pressed and when I asked the agent to know if it was great, it told me to use ===. I decided to follow the agent's advice.

This shows us that the inline suggestion makes suggestion but it's sometimes not interesting to use what it proposed you.

3. Reflection

My configuration turned Copilot into a tutor instead of a code generator. The quiz after each step forced me to check that I understood the concepts before moving on. Limiting the code the agent could give made the agent explain the problem and give hints, so I learned more from my own mistakes.

In this project I used let and const for variables, arrays with unshift, pop to move the snake. I also used if and else conditions, a while loop and a for loop to place the food, a switch for the arrow keys, and the DOM API with getElementById, textContent, addEventListener for the score and controls. New concepts for me were setInterval for

the game loop, localStorage for the high score. The agent helped me understand these concepts with hints and short quizzes.

I had the choice between inline suggestions and the chat agent, but they have different uses. The inline suggestions complete code quickly while I type, but they do not explain anything, I had to look by myself to get the explanation. The Chat agent follows my custom instructions.

Sometimes the agent gave me lines of code that were not useful. For example, when I told it that I had finished a step, it sent me the corrected code even though the step was already done. This pushed me to modify its configuration file again, so that it would stop giving code when I did not ask for it.

If I did this again, I would allow short code examples from the start, because forbidding all code slowed me down at the beginning. I would ask it to give hints in levels, from a small clue to a more detailed explanation. I would keep the quiz and the console debugging reminder, because they helped me a lot.

I would place GitHub Copilot as an assistant. I stayed in charge of every decision and every line of code, but it helped me when I was blocked and it confirmed my steps. Its position on the axis is the same as Copilot without a custom configuration, but I had more control over how it helps me.